МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №3

Дисциплина: ТЕХНОЛОГИИ СОЗДАНИЯ И ПОДДЕРЖКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Тема: «Разработка технического задания»

Индивидуальное задание: сервис Flowra – персональный планировщик задач.

Работу выполнила: _____ Баннова.Д.А

Направление подготовки: 09.03.03 Прикладная информатика

Преподаватель: профессор кафедры анализа данных и искусственного интеллекта факультета компьютерных технологий и прикладной математики, к. техн. н., доцент

_____ Р.Ю. Вишняков

Краснодар

2026

Цель работы:

Изучение теоретических основ и получение практических навыков создания, структурирования и проверки технических заданий (ТЗ) для программных продуктов на примере разрабатываемой интеллектуальной системы планирования задач Flowra

Введение:

Разработка любого качественного программного продукта невозможна без четкого понимания того, что именно необходимо создать. Если в первой лабораторной работе мы выявляли потребности пользователя с помощью неформальных пользовательских историй («хотелок») и вариантов использования, а во второй — проектировали внутреннюю структуру с помощью UML-моделей, то третья лабораторная работа посвящена связующему звену — Техническому заданию (ТЗ). Техническое задание выступает строгим юридическим и техническим документом. Это своеобразный «мост» между тем, чего ожидает конечный пользователь (или заказчик), и тем, что в итоге напишет программист. В данном отчете сформировано полноценное ТЗ на разработку сервиса Flowra — персонального планировщика, который с помощью математических алгоритмов и элементов искусственного интеллекта будет распределять задачи пользователя в зависимости от его биоритмов (хронотипа) и уровня усталости. Разрабатываемое ТЗ закладывает фундамент для дальнейшей работы над проектом: написания курсовой работы на 3-м курсе (где будут исследованы математические модели планирования) и последующей разработки кода, которая завершится защитой выпускной квалификационной работы (ВКР) на 4-м курсе.

Теоретическая часть:

1.1 Понятие технического задания

Техническое задание (ТЗ) — это основополагающий документ, содержащий набор требований заказчика к программной системе, в соответствии с которым осуществляется её проектирование, разработка, тестирование и внедрение. ТЗ нужно заказчику для того, чтобы понимать, за что он платит деньги и как проверить готовый результат. Разработчику ТЗ необходимо, чтобы четко спланировать архитектуру, выбрать стек технологий и избежать ситуации «сделай то, не знаю что».

1.2 Структура ГОСТ 34.602-2020

В Российской Федерации основным стандартом, регламентирующим состав ТЗ на автоматизированные системы, является ГОСТ 34.602-2020. Согласно ему, документ должен включать следующие разделы:

1. Общие сведения.
2. Назначение и цели создания (развития) системы.
3. Характеристики объектов автоматизации.
4. Требования к системе.
5. Состав и содержание работ по созданию системы.
6. Порядок контроля и приемки системы.
7. Требования к составу и содержанию работ по подготовке объекта к вводу в эксплуатацию.
8. Требования к документированию.
9. Источники разработки (при необходимости).

1.3 Отличия ГОСТ 34.602-1989 от ГОСТ 34.602-2020

Предыдущая версия стандарта (1989 года) была создана в эпоху больших ЭВМ и водопадной (Waterfall) модели разработки, когда программы создавались годами без промежуточных результатов. Новый ГОСТ 34.602-2020 был актуализирован с учетом современных реалий. Он стал более гибким, в нем лучше прописаны требования к информационной безопасности, облачным

технологиям и интеграции систем. Кроме того, новый стандарт допускает использование гибких методологий (Agile) и итеративную сдачу проекта, что отлично подходит для разработки мобильных и веб-приложений, таких как Flowra.

1.4. Характеристики правильного требования

Чтобы программист не понял задачу двояко, каждое требование в ТЗ должно обладать рядом характеристик:

- Необходимость: требование действительно решает проблему пользователя.
- Атомарность: одно требование описывает только одну функцию. (Плохо: Система должна сохранять задачи и отправлять СМС. Хорошо: 1. Система должна сохранять задачи. 2. Система должна отправлять СМС.)
- Понятность: текст трактуется однозначно. Независимость от реализации: мы пишем «что» должна делать система, а не «как» в коде. (Плохо: Система должна использовать цикл for для поиска задач. Хорошо: Система должна выполнять поиск задач по их названию.) Проверяемость: можно написать тест, который даст ответ «да/нет». (Плохо: Приложение должно работать быстро. Хорошо: Приложение должно загружаться не более чем за 3 секунды.)
- Правдоподобность: требование можно реализовать в рамках бюджета и технологий.

1.5. Функциональные и нефункциональные требования

Функциональные требования (ФТ) описывают поведение системы – что она конкретно делает. Например: «Пользователь должен иметь возможность удалить задачу». Это прямое действие.

Нефункциональные требования (НФТ) описывают атрибуты качества – как именно система выполняет свои функции. Это требования к безопасности (шифрование паролей), производительности (скорость ответа сервера), удобству использования (адаптивный дизайн) и надежности.

1.6. Идентификация требований

В крупных проектах требований сотни. Чтобы на них ссылаться в коде или тестах, каждому требованию присваивается уникальный идентификатор (ID). Например, префикс FR (Functional Requirement) используется для функциональных требований (FR-001), а NFR – для нефункциональных. Это позволяет легко найти нужное требование в таблице и не запутаться при изменениях.

1.7. Этапы подготовки ТЗ

Составление ТЗ — это не просто написание текста, а процесс, включающий: сбор и анализ требований (написание пользовательских историй), исследование конкурентов, определение целей проекта, детальную проработку ФТ и НФТ, выбор стека технологий (языков программирования), проектирование архитектуры (UML-диаграммы), составление плана-графика работ, определение критериев тестирования, оценку рисков и, наконец, итоговое документирование

ПРАКТИЧЕСКАЯ ЧАСТЬ — ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА СЕРВИС FLOWRA

2.1. Общие сведения о системе

Полное наименование системы: Интеллектуальный персональный планировщик задач Flowra.

Краткое наименование: ИС Flowra (или сервис Flowra).

Заказчик и Разработчик: Баннова Д.А., студентка КубГУ, направление 09.03.03 «Прикладная информатика». Плановые сроки выполнения работ:

начало — сентябрь 2026 г., окончание — июнь 2028 г. (защита выпускной квалификационной работы).

Источники финансирования: инициативный учебный проект (без привлечения коммерческого финансирования на этапе разработки прототипа).

2.2. Назначение и цели создания системы

Назначение: Система Flowra предназначена для автоматизации процесса персонального планирования времени (тайм-менеджмента) пользователей с учетом их индивидуальных биологических ритмов (хронотипа), фиксированных событий (учеба, работа) и уровня текущей энергетической нагрузки.

Цели создания:

- Сокращение времени, затрачиваемого пользователем на ручное составление расписания на день.
- Повышение личной продуктивности за счет распределения сложных когнитивных задач на пики активности.
- Снижение уровня стресса и прокрастинации благодаря механизму «Rescue Mode» (умному переносу задач при перегрузке).
- Измеримые критерии достижения целей: Время формирования плана на день не должно превышать 2 минут. Удовлетворенность пользователя предложенным расписанием (измеряемая отсутствием ручных правок) должна составлять не менее 70%

2.3. Характеристика объектов автоматизации

Объектом автоматизации является процесс распределения повседневных и рабочих задач человека (студента, фрилансера, специалиста). В классическом виде человек использует бумажные блокноты или простые приложения (To-Do листы), где задачи выстроены единым списком. Проблема такого подхода заключается в том, что пользователь часто не может объективно оценить свое свободное время и ставит себе больше задач, чем физически способен выполнить (возникает «овербукинг»). Кроме того,

выполнение сложных задач в период спада энергии приводит к быстрой утомляемости. Система Flowra должна автоматизировать этот процесс, взяв математические расчеты свободных слотов на себя.

2.4. Требования к системе

2.4.1. Требования к системе в целом Система Flowra представляет собой клиент-серверное приложение, состоящее из следующих логических модулей: клиентское приложение (мобильный и веб-интерфейс), серверная часть (API-шлюз и бизнес-логика), модуль математической оптимизации (алгоритмы расписания), модуль искусственного интеллекта (распознавание естественного языка) и база данных. Система должна функционировать в режиме 24/7. Целевая аудитория: лица в возрасте от 16 до 40 лет, нуждающиеся в инструментах тайм менеджмента.

2.4.2. Функциональные требования (ФТ) Ниже представлены основные функциональные требования, базирующиеся на пользовательских историях, разработанных в Лабораторной работе №1. Приоритет указан по методологии MoSCoW (Must — обязательно, Should — желательно, Could — возможно)

ID	Группа	Описание требования	Приоритет
FR 001	Управление задачами	Система должна позволять пользователю создавать новую задачу с указанием названия, длительности и крайнего срока (дедлайна).	Must
FR 002	Управление задачами	Система должна позволять указывать тип задачи («глубокая работа» или «рутина») и приоритет от 1 до 5.	Must
FR 003	Управление задачами	Система должна позволять редактировать атрибуты существующей задачи.	Must

FR 004	Управление задачами	Система должна позволять отмечать задачу как выполненную, обновляя её статус в базе данных.	Must
FR 005	Управление задачами	Система должна уметь автоматически разбивать задачи длительностью более 4 часов на несколько подзадач.	Should
FR 006	Профиль	Система должна проводить опрос пользователя при регистрации для вычисления его хронотипа (жаворонок, сова, голубь).	Must
FR 007	Профиль	Система должна позволять пользователю задавать жесткое время отхода ко сну и пробуждения.	Must
FR 008	Профиль	Система должна формировать "кривую энергии" пользователя на графике в зависимости от выбранного хронотипа.	Must
FR 009	Профиль	Система должна позволять изменять настройки профиля с последующим пересчетом активного расписания.	Should
FR 010	События	Система должна позволять добавлять "фиксированные события" (например, пары в университете), которые нельзя сдвигать.	Must
FR 011	События	Система должна поддерживать создание повторяющихся событий (еженедельно, по четным/нечетным неделям).	Must
FR 012	События	Система должна уметь импортировать фиксированные события из файлов формата .ics (календари Google, Apple).	Could

FR 013	Расписание	Система должна по нажатию кнопки генерировать расписание на день, распределяя активные задачи по свободным временным слотам.	Must
FR 014	Расписание	Система должна размещать задачи типа "глубокая работа" в те часы, где "кривая энергии" пользователя достигает максимума.	Must
FR 015	Расписание	Система не должна назначать задачи на время сна или во время фиксированных событий.	Must
FR 016	Расписание	Система должна позволять пользователю вручную перетаскивать (drag-and-drop) сгенерированные блоки задач по шкале времени.	Should
FR 017	Адаптация	Система должна предоставлять кнопку "Я не успел", которая запускает пересчет расписания на оставшуюся часть дня.	Must
FR 018	Адаптация	Система должна вычислять метрику загруженности дня в процентах.	Should
FR 019	Адаптация	Система должна активировать "Rescue Mode" при перегрузке (>85%), автоматически перенося низкоприоритетные задачи на завтра.	Must
FR 020	ИИ-модуль	Система должна принимать текст задачи в свободной форме (естественный язык).	Should

FR 021	ИИ-модуль	Система должна извлекать из текста название, длительность и дедлайн, используя модули NLP (обработки естественного языка).	Should
FR 022	ИИ-модуль	Система должна отображать предзаполненную карточку задачи после анализа текста для подтверждения пользователем.	Should
FR 023	Аналитика	Система должна рассчитывать процент выполненных задач за неделю.	Should
FR 024	Аналитика	Система должна показывать график "План против Факта" (запланированное время против реального).	Could
FR 025	Уведомления	Система должна отправлять push-уведомление за 15 минут до начала запланированной задачи.	Must
FR 026	Уведомления	Уведомление должно содержать кнопки быстрого действия: "Начал", "Отложить на 15 мин".	Should

2.4.3. Нефункциональные требования (НФТ)

Категория	ID	Описание требования
Производительность	NFR 001	Время генерации расписания на неделю алгоритмом оптимизации не должно превышать 5 секунд.
Надёжность	NFR 002	Доступность (uptime) серверной части должна составлять не менее 99.0% времени в месяц.

Безопасность	NFR 003	Пароли пользователей должны храниться в базе данных в виде хэшей с использованием алгоритма bcrypt.
Безопасность	NFR 004	Обмен данными между клиентом и сервером должен осуществляться по защищенному протоколу HTTPS. Авторизация — через JWT токены.
Удобство (UI/UX)	NFR 005	Пользовательский интерфейс должен быть адаптивным. Доступ к функции добавления задачи должен осуществляться не более чем за 2 клика с главного экрана.
Масштабируемость	NFR 006	Архитектура базы данных должна поддерживать стабильную работу при объеме до 10 000 активных пользователей.
Совместимость	NFR 007	Мобильное приложение должно поддерживать ОС Android 10+ и iOS 15+.

2.5. Требования к видам обеспечения

Математическое обеспечение: ядром системы является алгоритм генерации расписания. Будут использоваться эвристические алгоритмы и методы смешанного целочисленного линейного программирования (MILP) для решения задачи упаковки в контейнеры (Bin Packing Problem) с учетом временных ограничений. Детальный анализ этих алгоритмов и формул станет предметом курсовой работы в 6 семестре (3 курс).

Информационное обеспечение: реляционная структура базы данных, включающая основные таблицы: Users (пользователи), Tasks (задачи), FixedEvents (события), ScheduleSlots (слоты расписания) и Analytics (история действий).

Лингвистическое обеспечение: интерфейс системы должен поддерживать русский язык (по умолчанию) с возможностью расширения до

английского. Взаимодействие ИИ-ассистента с пользователем также осуществляется на русском языке.

Программное обеспечение (стек технологий): Мобильная разработка: кроссплатформенный фреймворк Flutter (язык Dart) или React Native. Backend (серверная логика): Python (фреймворк FastAPI). Выбор Python обусловлен простотой интеграции математических библиотек и ИИ-модулей. СУБД: PostgreSQL (для надежного хранения) и Redis (в качестве кэша и очереди задач для рассылки уведомлений). ИИ-модуль: интеграция с OpenAI API или локальной NLP моделью (например, spaCy/DeepPavlov).

Техническое обеспечение: для серверной части требуется виртуальный выделенный сервер (VPS) с характеристиками не ниже 2 vCPU, 4 GB RAM, 40 GB NVMe. Для разработки — персональный компьютер/ноутбук на базе ОС Windows/macOS.

2.6. Состав и содержание работ по созданию системы

Разработка системы Flowra планируется в рамках учебного процесса (до получения диплома бакалавра) и разбита на следующие макро-этапы:

- Этап 1: Исследование и математическое моделирование (сентябрь 2026 январь 2027). Изучение алгоритмов теории расписаний, разработка математической модели биоритмов, защита курсовой работы.
- Этап 2: Разработка MVP (февраль 2027 – июнь 2027). Создание серверного ядра на Python, проектирование базы данных, реализация базового алгоритма генерации расписания без ИИ.
- Этап 3: Расширение функционала (сентябрь 2027 – февраль 2028). Разработка мобильного интерфейса (фронтенд), подключение модуля обработки естественного языка (ИИ-ассистент), реализация Rescue Mode.
- Этап 4: Тестирование и отладка (март 2028 – май 2028). Проведение юнит тестирования, интеграционного тестирования,

устранение багов, написание текста выпускной квалификационной работы.

- Этап 5: Сдача в эксплуатацию (июнь 2028). Защита ВКР перед государственной экзаменационной комиссией.

2.7. Порядок контроля и приёмки системы

Контроль осуществляется путем демонстрации рабочих сценариев. Виды испытаний: автономные (тестирование отдельных функций алгоритма), комплексные (работа всей системы в связке клиент-сервер) и приемочные. Приемочные испытания считаются успешными, если система может: а) зарегистрировать пользователя, б) принять 5 задач различной сложности, в) за время <5 секунд сформировать корректное расписание без пересечения с событиями.

2.8. Подготовка к вводу в эксплуатацию

Перед защитой проекта необходимо подготовить тестовый стенд (развернуть сервер на хостинге), создать тестовый аккаунт с предварительно заполненными данными для демонстрации комиссии и, по возможности, опубликовать клиентскую часть в виде PWA (прогрессивного веб-приложения) или APK-файла.

2.9. Требования к документированию

По итогам работы должны быть подготовлены: пояснительная записка ВКР (описывающая архитектуру, математику и код), руководство пользователя (инструкция по эксплуатации приложения) и файл README для репозитория исходного кода.

3. ОЦЕНКА РИСКОВ И ОГРАНИЧЕНИЙ

Поскольку разработчик является студентом (новичком в коммерческой разработке) и проект разрабатывается в условиях ограниченного времени, необходимо заранее предусмотреть риски.

Описание риска	Вероятность	Влияние	Стратегия смягчения (минимизации)
Нехватка времени на реализацию ИИ-модуля (NLP) из-за высокой сложности.	Высокая	Среднее	Снизить приоритет требования до "Should". Вместо обучения собственной модели использовать готовое стороннее решение (OpenAI API или Yandex API).
Алгоритм оптимизации (MILP) будет работать слишком долго при большом числе задач.	Средняя	Высокое	Разработать запасной (резервный) "жадный" алгоритм планирования. Если MILP не находит ответ за 3 секунды, система переключается на простой быстрый алгоритм.

Сложности с кроссплатформенной мобильной разработкой (Flutter).	Средняя	Среднее	Начать разработку с простого веб интерфейса (React/Vue) или Telegram бота. Мобильное приложение реализовать при наличии остатка времени.
Отсутствие опыта в настройке серверов Linux.	Низкая	Среднее	Использовать облачные PaaS платформы, которые берут часть настройки на себя (например, Heroku или Яндекс.Облако Serverless).

4. РАЗДЕЛ С UML-ДИАГРАММАМИ

В данном разделе (в дополнение к ним) проектируются две новые структурные диаграммы, необходимые для технического задания: диаграмма развертывания и диаграмма пакетов.

4.1. Диаграмма развертывания (Deployment Diagram)

Данная диаграмма показывает физическое расположение компонентов программного обеспечения на аппаратных узлах. Она отвечает на вопрос «На каком компьютере/сервере работает та или иная часть системы?». В нашем

случае система Flowra распределена между смартфоном пользователя и облачным сервером.

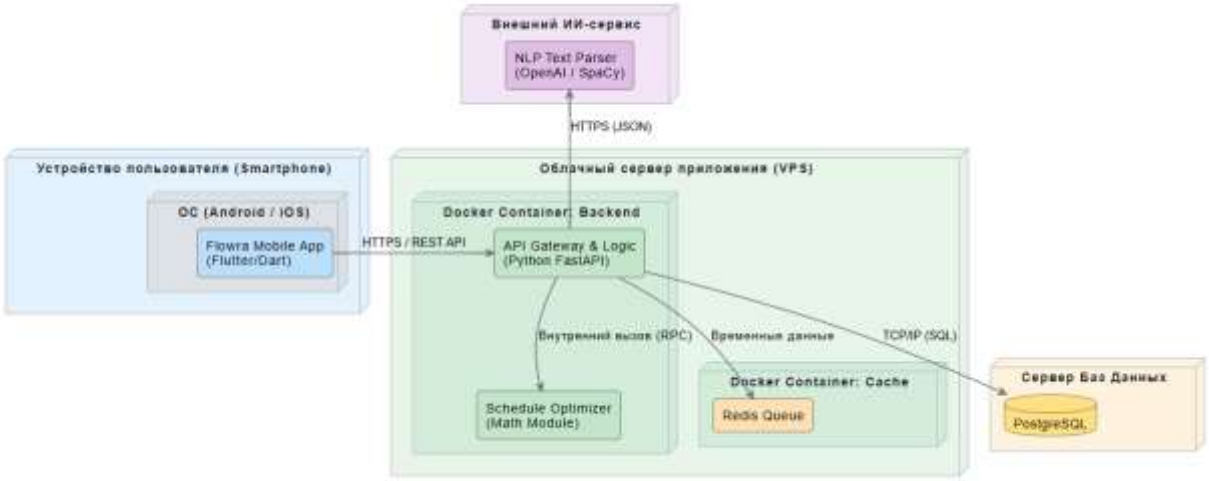


Рисунок 1 — Диаграмма развертывания физической архитектуры Flowra

- **Уровень клиента (Client Layer):** Представлен мобильным устройством под управлением Android или iOS. В качестве фронтенда используется кроссплатформенное приложение на фреймворке **Flutter**. Это обеспечивает единую кодовую базу и высокую скорость работы интерфейса. Взаимодействие с сервером происходит по защищенному протоколу HTTPS через REST API.
- **Серверный уровень (Application Server):** Серверная часть развернута на виртуальном выделенном сервере (VPS) с использованием технологии контейнеризации **Docker**. Это позволяет изолировать зависимости и упростить масштабирование.
    - **Backend (FastAPI):** Основной узел системы на языке Python. Он отвечает за аутентификацию, обработку запросов и бизнес-логику.
    - **Schedule Optimizer:** Выделенный математический модуль. Он вынесен в отдельный блок, так как алгоритм оптимизации расписания (MILP) является ресурсоемким процессом.

    - **Redis:** Используется как кэш и очередь задач для промежуточного хранения данных, что повышает отклик системы.
- **Уровень данных (Data Layer):** Используется реляционная СУБД **PostgreSQL**. Она выбрана за надежность и поддержку сложных связей между задачами, событиями и профилями пользователей. Связь с бэкендом осуществляется по протоколу TCP/IP.
- **Внешние интеграции:** Система взаимодействует с внешним ИИ-сервисом (например, OpenAI) для парсинга текста задач на естественном языке (NLP). Это позволяет не перегружать сервер сложными нейросетевыми вычислениями.

4.2. Диаграмма пакетов (Package Diagram)

Диаграмма пакетов показывает, как исходный код бэкенда разделен на логические папки (модули). Грамотное разделение на пакеты предотвращает превращение кода в «спагетти». Мы используем чистую архитектуру: пользовательский интерфейс (роутеры) отделен от бизнес-логики и базы данных.

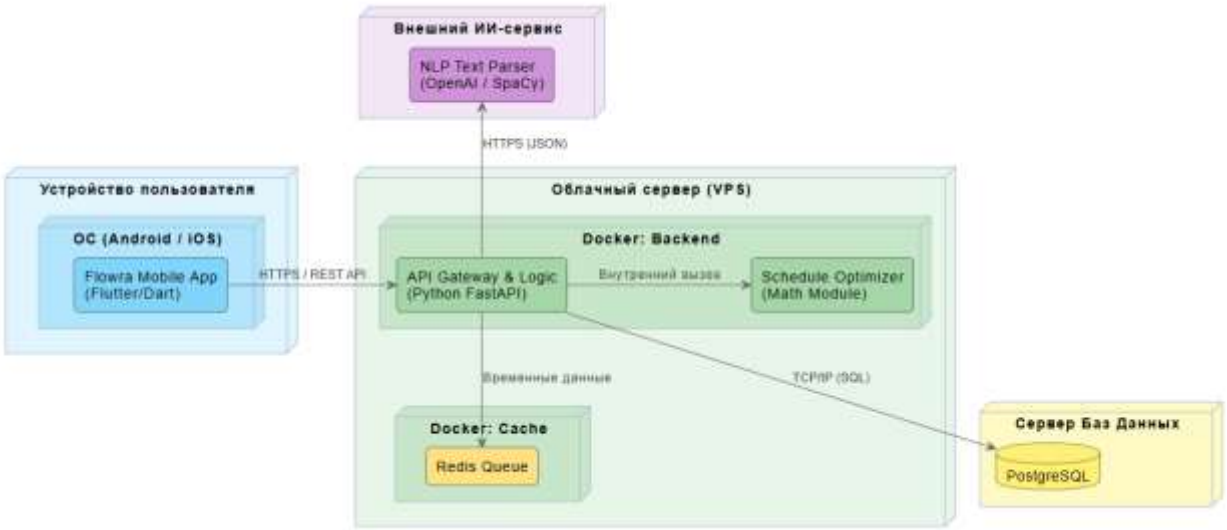


Рисунок 2 — Диаграмма пакетов (структура исходного кода серверной части)

В этом разделе описывается, как компоненты, указанные на схеме, работают вместе для реализации ключевых функций:

1. **Процесс создания задачи:**
    - Пользователь вводит текст в мобильном приложении.
    - Приложение отправляет запрос на **Backend**.
    - Backend перенаправляет текст во **внешний ИИ-сервис (NLP)**, получает структурированные данные (название, дедлайн, длительность) и возвращает их пользователю для подтверждения.
    - После подтверждения данные записываются в базу данных **PostgreSQL**.
2. **Процесс генерации расписания:**
    - При нажатии кнопки «Сгенерировать» Backend запрашивает из базы данных все активные задачи и фиксированные события пользователя.
    - Данные передаются в **Schedule Optimizer**, который проводит расчеты, учитывая «кривую энергии» и приоритеты.
    - Готовое расписание кэшируется в **Redis** для быстрого доступа и сохраняется в основную базу.
    - Мобильное приложение получает обновленный план через API и отрисовывает его на таймлайне.
3. **Обеспечение отказоустойчивости:**
    - Разделение на Docker-контейнеры гарантирует, что если модуль оптимизатора будет перегружен сложными расчетами, основной API Gateway (Backend) продолжит принимать запросы от других пользователей, обеспечивая высокую доступность (Uptime), указанную в требованиях NFR.

5. ПРОВЕРКА ТРЕБОВАНИЙ НА СООТВЕТСТВИЕ ХАРАКТЕРИСТИКАМ

Для контроля качества составленного ТЗ, проанализируем 5 случайно выбранных функциональных требований по 6 критериям качества. Символ «+» означает, что требование соответствует критерию.

ID / Требование	Необходимость	Атомарность	Понятн.	Независимость от реализации	Проверяемость	Правдоподобность
FR-004: Отметка задачи как выполненной	+	+ (одно действие)	+	+ (не указан метод ввода)	+ (статус в БД меняется на Done)	+
FR-008: Формирование "кривой энергии"	+	+	+	+	+ (график виден на экране)	+
FR-015: Запрет на задачи во время сна	+	+	+	+	+ (тест на пересечение слотов)	+
FR-021: Извлечение данных из текста (NLP)	+	+	+	+ (не привязан к провайдеру ИИ)	+ (сравнение ввода и JSON выхода)	+ (реализуемо через API)
NFR-001: Генер	+	+	+	+	+ (замеряе	+ (адекватно

ация расписания до 5 сек.					тся таймером)	для бэкенда)

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое техническое задание? Техническое задание (ТЗ) — это официальный юридически значимый и технический документ, содержащий полный перечень требований заказчика к программной системе, а также определяющий порядок её создания, тестирования и внедрения.

2. Для кого составляется техническое задание? ТЗ является двусторонним документом. Оно составляется для заказчика (чтобы он видел, что его требования учтены и знал, как проверять результат) и для разработчика (программистов, тестировщиков, архитекторов — чтобы они понимали, какой продукт нужно спроектировать, какие технологии использовать и какие ограничения соблюдать).

3. Может ли техническое задание на разработку системы не содержать требований к тестированию, приемке? Если да, то в каком случае? В строгом соответствии с ГОСТ 34.602-2020 эти разделы обязательны. Однако на практике, если ТЗ пишется для внутреннего использования стартапом на стадии создания быстрого прототипа (MVP) или проверки гипотезы, раздел приемки может быть упущен для экономии времени. Также он может быть вынесен в отдельный документ — "Программу и методику испытаний" (ПМИ).

4. Какие основные блоки входят в построение технического задания? К основным блокам относятся: общие сведения, назначение и цели, характеристика объекта автоматизации, функциональные и нефункциональные требования к системе, требования к видам обеспечения (информационное, программное), этапы работ, порядок контроля и приемки, требования к документации.

5. Какие существуют этапы подготовительных работ по созданию технического задания? Подготовительные этапы включают: сбор первичных требований (интервью, создание пользовательских историй), анализ рынка и конкурентов, определение целей проекта, разделение требований на функциональные и нефункциональные, черскизирование архитектуры (UML) и оценку рисков.

6. Может ли техническое задание быть составлено не в соответствии с ГОСТ? Да, может. Если разработка ведется для частной коммерческой компании или зарубежного заказчика, ТЗ может составляться в свободном формате, по стандарту IEEE 29148 (Software Requirements Specification - SRS) или вовсе в виде набора карточек в Jira (в рамках методологии Agile). Строгое соблюдение ГОСТ обязательно при работе с государственными заказчиками РФ.

7. Приведите пример структуры технического задания по ГОСТ 34.602-2020. 1. Общие сведения. 2. Назначение и цели. 3. Характеристики объектов автоматизации. 4. Требования к системе. 5. Состав и содержание работ. 6. Порядок контроля и приемки. 7. Требования к подготовке ввода в эксплуатацию. 8. Требования к документированию.

8. В чем основное структурное отличие ГОСТ 34.602-1989 и ГОСТ 34.602-2020? ГОСТ 2020 года адаптирован под современные IT-реалии. В нем добавлены и расширены требования к информационной безопасности, защите персональных данных, использованию облачных вычислений. Также он допускает поэтапную (итерационную) разработку и сдачу системы по частям, что соответствует современным Agile-подходам.

9. Почему необходимо уметь составлять техническое задание? Грамотное ТЗ минимизирует риски проекта. Оно защищает разработчика от бесконечных правок со стороны заказчика ("я имел в виду другое"), позволяет точно оценить сроки и стоимость разработки, а также служит основой для написания автотестов и технической документации.

10. Должны ли требования соответствовать определенным характеристикам? Если да, то каким? Да, любое качественное требование должно обладать следующими характеристиками: необходимость, атомарность (одно требование — одна функция), понятность, независимость от реализации, проверяемость (тестируемость) и правдоподобность (реалистичность).

11. Для чего необходима идентификации требований в техническом задании? Идентификация (присвоение уникальных номеров, например, FR-01) нужна для того, чтобы на требование можно было ссылаться из других документов, отслеживать его реализацию в коде, привязывать к конкретным тестам (трассируемость требований) и легко управлять изменениями без путаницы.

12. Приведите примеры систем идентификации, назовите их сильные и слабые стороны.

Простая числовая (1, 2, 3...): Сильные стороны: простота. Слабые: при добавлении нового пункта в середину вся нумерация съезжает.

Иерархическая (1.1, 1.1.2): Сильные стороны: отражает структуру. Слабые: номера становятся длинными и тяжело читаемыми, съезжают при изменениях структуры.

Мнемоническая (FR-001, SEC-005): Сильные стороны: номер уникален, не зависит от положения в тексте, префикс сразу говорит о типе требования (Security, Functional). Слабые: требует ведения отдельного реестра кодов.

UUID (уникальный хэш): Сильные стороны: 100% уникальность. Слабые: абсолютно нечитаемо для человека (например, 123e4567-e89b-12d3...)

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были успешно освоены теоретические и практические аспекты создания технического задания на разработку программного обеспечения. На базе ранее сформированных пользовательских историй и UML-моделей был разработан полноценный документ ТЗ для интеллектуального планировщика задач Flowra, структурированный в соответствии с ГОСТ 34.602-2020. Сформулированы конкретные, измеримые функциональные и нефункциональные требования, проведена их проверка на соответствие критериям качества (атомарность, проверяемость), разработаны диаграммы развертывания и пакетов. Данное ТЗ выступает прочным фундаментом для последующих этапов: написания курсовой работы (исследование математических алгоритмов оптимизации) и, в перспективе, реализации и успешной защиты выпускной квалификационной работы.